

NIFTY-50 Investment Intelligence Platform

Technical Report

Dataset: NIFTY-50 Stock Market Data (Kaggle), 2000–2021

Stack: Python · pandas · scikit-learn · XGBoost · SciPy · Streamlit

Repository: <https://github.com/rakshit11147/nifty-50-investment-intelligence>

1. Exploratory Data Analysis (EDA)

The dataset contains daily OHLCV (Open, High, Low, Close, Volume) records for all NIFTY-50 constituents from 2000 to 2021, with one CSV file per stock.

Steps performed (notebooks/01_EDA.py):

- **Data quality checks:** missing values, duplicate dates, and inconsistent symbols were identified and cleaned. Dates were parsed and set as a sorted index per stock.
- **Price trends:** long-term close-price trajectories were plotted per stock, showing strong secular growth in IT (TCS, INFY) and banking (HDFCBANK) names, and high cyclicalities in metals and energy.
- **Return distributions:** daily returns are non-normal, exhibiting fat tails and negative skew. Extreme moves cluster around crisis periods (2008 global financial crisis, 2020 COVID crash).
- **Volatility clustering:** rolling 20-day volatility confirms that high-volatility periods cluster in time, motivating volatility-based features.
- **Correlation analysis:** a cross-stock return correlation heatmap shows strong intra-sector correlation (banks with banks, IT with IT), which justifies diversification across sectors in portfolio construction.

***Key takeaway:** prices are non-stationary, but returns and indicator-based ratios are usable as model inputs. Direction (up/down) and short-horizon returns are more learnable targets than the exact future price level — which is why the platform also presents probabilistic price ranges (Section 4) rather than a single forecast.*

2. Feature Engineering

All features are computed in `src/utils/indicators.py` (`TechnicalIndicators.add_all()`), configured through `config/settings.py`.

Feature group	Features	Purpose
Trend	SMA 20/50/200, EMA 12/26, Price_vs_SMA20/50	Trend position relative to moving averages
Momentum	MACD, MACD_Signal, MACD_Hist, RSI_14, ROC_10, Momentum_5/20	Speed and persistence of moves
Volatility	Bollinger %B and Width, ATR_14 (% of price), rolling 20/50-day volatility	Regime and range information
Volume	OBV (On-Balance Volume)	Volume confirmation of price moves
Price structure	Daily log return, High/Low ratio, Open/Close ratio	Intraday behavior

Design choices:

- Ratios and normalized features (e.g., $\text{Price_vs_SMA20} = \text{Close/SMA20} - 1$, ATR_Pct) are used instead of raw prices so features are comparable across stocks and time, and stationary enough for tree models.

- Rows with NaN values (from rolling windows) are dropped after computation.
- The final model feature set contains 15 features (StockPredictor.FEATURE_COLS).

3. Methodology

The overall pipeline is:

```
Raw OHLCV -> Cleaning -> Technical indicators -> Feature matrix X
-> Direction label: 1 if Close[t+h] > Close[t], else 0 (h = configurable horizon, default 10)
-> Forward-return label: (Close[t+h] / Close[t]) - 1
-> Chronological train/test split (80/20, no shuffle)
-> Train RF + XGBoost (direction) and XGBoost Regressor (forward return)
-> Evaluate (classification metrics + MAE/RMSE/R2/Directional Accuracy)
-> Explain (gain-based feature importances)
-> Monte Carlo simulation for probabilistic price range
```

Prediction targets:

- *Direction*: binary classification of whether the close price is higher h days ahead than today (h is user-configurable in the dashboard, default 10).
- *Forward return*: the XGBoost Regressor predicts the h-day forward return directly, which is converted to an implied target price using the last known close.
- **Train/test split**: strictly chronological (first 80% train, last 20% test) to avoid look-ahead bias. Data is never shuffled.
- **Feature scaling**: MinMax scaling applied where required (price-based scaling for Monte Carlo calibration).
- **Evaluation metrics — direction models (RF, XGBoost Classifier)**: accuracy, per-class precision/recall/F1 (classification report), a confusion matrix on the last 100 test samples, and a rolling 20-sample accuracy chart benchmarked against the 50% coin-flip baseline.
- **Evaluation metrics — forward-return regressor**: MAE, RMSE, and R2 (on implied price level), plus Directional Accuracy (sign of predicted vs. actual return).
- **Persistence**: trained models are serialized with joblib to models_saved/.

4. Model Architecture

Implemented in src/models/predictor.py (StockPredictor class).

Model	Task	Key hyperparameters
Random Forest	h-day direction classification	200 trees, max_depth = 10, random_state = 42
XGBoost Classifier	h-day direction classification	200 estimators, max_depth = 6, learning_rate = 0.05, subsample = 0.8
XGBoost Regressor	h-day forward return / price forecast	300 estimators, max_depth = 5, learning_rate = 0.05, subsample = 0.8, random_state = 42
Monte Carlo Simulation	Probabilistic price range	500 Geometric Brownian Motion paths; drift (mu) and volatility (sigma) estimated from historical daily returns

- All three ML models consume the same 15-feature matrix derived from TechnicalIndicators.
- XGBoost's regularized gradient boosting captures nonlinear feature interactions; Random Forest provides a stable, lower-variance baseline for direction classification.

- The **XGBoost Regressor** produces a single point estimate for the h-day forward price ("ML target"), which is displayed as one marker on top of the Monte Carlo distribution rather than as a standalone forecast.
- The **Monte Carlo module** is a statistical complement to the regressor: it simulates 500 future price paths using the stock's historically estimated drift and volatility under Geometric Brownian Motion, producing 50% and 90% confidence bands plus pessimistic (5%), expected (median), and optimistic (95%) price ranges for each day of the horizon. This directly implements the "decision support, not point prediction" requirement — the dashboard shows a range of plausible outcomes with the ML estimate positioned inside it, rather than a single confident number.
- The engine is symbol-agnostic: one StockPredictor instance is trained per stock (e.g., `StockPredictor(symbol='RELIANCE')`).

5. Portfolio Construction Logic

Implemented in `src/models/portfolio.py` (PortfolioBuilder class) using mean-variance optimization solved with SciPy's SLSQP optimizer.

Inputs: daily close prices of the selected stocks -> daily returns -> annualized mean return vector and covariance matrix (252 trading days). Risk-free rate: 6% (India 10Y G-Sec proxy).

Three investor profiles, three objectives:

Profile	Objective	Target return	Max volatility
Conservative	Minimize portfolio volatility	8%	12%
Balanced	Maximize Sharpe ratio	15%	20%
Aggressive	Maximize expected return	25%	35%

Constraints applied to every optimization:

- Weights sum to 1 (fully invested)
- Per-stock bounds of 2% to 40% to enforce diversification and prevent corner solutions
- Long-only (no short selling)

6. Risk Assessment Methodology

Implemented in `src/models/risk.py` (RiskAssessor class). All metrics are annualized with 252 trading days and a 6% risk-free rate.

Metric	Definition
Historical Volatility	Std. dev. of daily returns x $\sqrt{252}$
Sharpe Ratio	$(\text{Annual return} - \text{risk-free}) / \text{volatility}$
Sortino Ratio	$(\text{Annual return} - \text{risk-free}) / \text{downside volatility only}$
Calmar Ratio	$\text{Annual return} / \text{max drawdown} $
Maximum Drawdown	Largest peak-to-trough decline of the cumulative value
VaR (95%)	Worst expected daily loss at 95% confidence; historical (empirical 5th percentile) and parametric (normal z-score) methods
CVaR / Expected Shortfall	Mean loss in the tail beyond VaR
Beta (optional)	Sensitivity vs. a NIFTY-50 index proxy; computed by <code>RiskAssessor.assess_stock()</code> when a benchmark return series is supplied

Note on Beta: the RiskAssessor class supports Beta calculation against any benchmark return series passed via the `market_returns` parameter. In the current dashboard, the Risk Assessment page does not yet pass a benchmark series, so Beta is not displayed in the UI. It remains available for command-line/notebook use and as a planned dashboard enhancement.

7. Explainability Techniques

- **Feature importances:** the dashboard displays native XGBoost gain-based `feature_importances_` for both the direction classifier and the forward-return regressor, ranked from most to least influential.

- **Direction model transparency:** beyond accuracy, the dashboard shows a confusion matrix (last 100 test samples) and a rolling 20-sample accuracy chart against the 50% coin-flip baseline, so users can visually assess whether a model is meaningfully better than random.
- **Transparent metrics:** every risk and portfolio figure shown in the UI maps to a documented formula in `risk.py` / `portfolio.py`, so results are auditable rather than black-box.
- **Probabilistic framing:** the Monte Carlo price-range chart is itself an explainability tool — it visually communicates the uncertainty around the ML point estimate rather than presenting it as a guaranteed outcome.

Across stocks, momentum (RSI, MACD) and volatility-based features (Price_vs_SMA50, Volatility_20, ATR_Pct) consistently rank as the strongest drivers of both direction and forward-return predictions.

8. Key Insights

1. Direction and short-horizon returns are more tractable than exact prices.

The platform frames forecasting primarily as direction classification and forward-return regression rather than exact price-level prediction, and supplements both with a Monte Carlo probability range so users see uncertainty explicitly.

2. Momentum and volatility dominate feature importance.

Price_vs_SMA50, MACD, Volatility_20, and ATR_Pct consistently rank as the top features across stocks, confirming that trend-position and volatility-regime information are the most useful signals available in this feature set.

3. Diversification is measurable.

Optimized portfolios materially reduce volatility versus single stocks at comparable returns; the Conservative profile cuts volatility well below the index while preserving most of the return.

4. Risk profiles matter.

The same stock universe produces very different allocations across Conservative/Balanced/Aggressive profiles, demonstrating that investor constraints, not just expected returns, drive allocation.

5. Fat tails are real.

Historical VaR consistently exceeds parametric VaR in magnitude, confirming that normal-distribution assumptions understate Indian equity tail risk; CVaR is reported for a more honest tail estimate.

6. Honest evaluation.

Chronological splits, no shuffling, and out-of-sample testing were enforced throughout to avoid look-ahead bias, a common pitfall in financial ML.

7. Near-random-walk behavior at short horizons, reported honestly.

On held-out data, the direction classifiers and the forward-return regressor's directional accuracy hover close to 50% — statistically near a coin flip. This is consistent with the efficient-market hypothesis at short horizons. Rather than overstating model skill, the platform leads with the Monte Carlo probability range and risk metrics, and presents the ML point estimate as one plausible scenario among many. The regressor's R2 on price level can appear high (driven largely by strong day-to-day price autocorrelation) and should always be read alongside directional accuracy, which is the more meaningful measure of genuine predictive skill.

End of report.